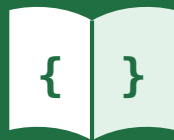

Module 19 — Native Image and Performance

Lesson Notes

Spring Boot Edu



2026-09-25

Contents

1. Learning Goals	2
2. Concepts	2
2.1 Why Startup Time Matters	2
2.2 What Happens at Startup, and What Can Move to Build Time	2
2.3 The Closed-World Assumption	3
3. Step-by-Step Examples	3
3.1 The Application	3
3.2 AOT Processing and AOT Repositories	4
3.3 Where AOT Is Not Enough: Dynamic Code	6
3.4 RuntimeHints	6
3.5 Building the Native Image	7
3.6 The JVM AOT Cache and the Comparison	8
4. Common Mistakes and Best Practices	9
5. Summary	10
6. Further Reading	10

1. Learning Goals

By the end of this module you will be able to:

- Explain what Spring's AOT processing does at build time, and read the generated code
- Use Spring Data AOT repositories, and know why AOT processing needs some configuration without a database
- Write RuntimeHints for reflection and resources, and test them
- Build a GraalVM native image with Paketo buildpacks, without installing GraalVM
- Speed up a normal JVM start with the JDK's AOT cache (Project Leyden)
- Measure startup time and memory, and choose the right variant for a workload

Prerequisites: Module 03 (Web MVC), Module 05 (Spring Data JDBC) · **Estimated time:** 4 hours · **Docker required**

2. Concepts

2.1 Why Startup Time Matters

A long-running server that starts once a week does not care about 3 seconds. But in some environments the start is on the critical path: scale-out under load, scale-to-zero (serverless), short batch jobs, CLI tools and fast feedback in development. Memory matters when many instances run side by side.

2.2 What Happens at Startup, and What Can Move to Build Time

On a normal start, Spring reads the configuration classes, evaluates conditions, creates bean definitions with reflection, and Spring Data derives queries from method names. All of this is the same on every start.

Technique	What moves to build time	Runs on
Spring AOT (process-aot)	bean definitions as generated Java code, repository implementations, hints	JVM or native
JVM AOT cache (JEP 483, 514, 515)	loaded and linked classes, method profiles from a training run	JVM (JDK 25+)

Technique	What moves to build time	Runs on
GraalVM native image	the whole application, compiled to machine code (closed world)	native executable

2.3 The Closed-World Assumption

native-image analyses the application at build time and includes only what it can reach. Code that is reached dynamically (reflection by class name, resources, proxies, serialization) is invisible to that analysis. It must be declared as **reachability metadata**. Spring's AOT engine writes most of this metadata by itself (for beans, controllers, Jackson types of controller methods). For your own dynamic code, you write RuntimeHints.

i Note

SPEC decision 8: there is no GraalVM for JDK 27 yet. This module is compiled with `maven.compiler.release=25`: the same classes run on JDK 27 (JVM, AOT cache), and GraalVM 25 turns them into a native image.

3. Step-by-Step Examples

With Docker running, start the application on the JVM. PostgreSQL starts from the root `compose.yaml`:

```
./mvnw -pl modules/19-native-performance/lesson spring-boot:run
```

The tour prints how the application runs:

```
=== mode: JVM, ready in 1402 ms ===
  Designing Data-Intensive Applications – €110,00
  Effective Java – €89,90
  Spring in Action – €95,00
```

3.1 The Application

A small bookstore API: GET `/api/books?title=...`, GET `/api/books/{isbn}`, GET `/api/books/{isbn}/price`, GET `/api/books/since/{year}` and GET `/api/quote` (see requests.http). The same jar is measured on the JVM, with the AOT cache and as a native image.

The tour reads the mode from Spring:

```

@EventListener
void ready(ApplicationReadyEvent event) {
    String mode = NativeDetector.inNativeImage() ? "native image"
        : AotDetector.useGeneratedArtifacts() ? "JVM with Spring AOT" : "JVM";
    Duration startup = event.getTimeTaken();
    System.out.println("=== mode: " + mode + ", ready in " + startup.toMillis() + " ms
        ↪ ===");
    books.publishedSince(2017).forEach(book ->
        System.out.println(" " + book.title() + " - " + prices.format(book.price())));
}

```

3.2 AOT Processing and AOT Repositories

The lesson binds process-aot into every build, not only into the native profile:

```

<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
  <executions>
    <!-- AOT processing in every build (not only with -Pnative): the jar can then
        ↪ also run
           on the JVM with -Dspring.aot.enabled=true, and AotProcessingIT checks the
        ↪ output -->
    <execution>
      <id>process-aot</id>
      <goals>
        <goal>process-aot</goal>
      </goals>
    </execution>
  </executions>
  <configuration>
    <image>
      <name>springbootedu/native-performance:${project.version}</name>
      <env>
        <BP_JVM_VERSION>25</BP_JVM_VERSION>    <!-- GraalVM / Liberica NIK 25 -->
      </env>
    </image>
  </configuration>
</plugin>

```

After `./mvnw -pl modules/19-native-performance/lesson` package, look into `lesson/target/spring-aot/main/`:

```

sources/com/springbootedu/nativeperformance/
  NativePerformanceApplication__BeanFactoryRegistrations.java    all bean definitions, as
↪ code
  book/BookController__BeanDefinitions.java                    "new BookController(...)"
↪ instead of reflection
  book/BookRepositoryImpl__AotRepository.java                  the repository's queries,
↪ as code
resources/META-INF/native-image/com.springbootedu/19-native-performance-lesson/
  reachability-metadata.json                                    reflection and resource
↪ metadata for GraalVM

```

The repository interface stays as usual:

```

public interface BookRepository extends ListCrudRepository<Book, String> {

    List<Book> findByTitleContainingIgnoreCaseOrderByTitle(String part);

    @Query("SELECT * FROM book WHERE year >= :year ORDER BY title")
    List<Book> publishedSince(int year);
}

```

With AOT, Spring Data does not derive the SQL of `findByTitleContainingIgnoreCaseOrderByTitle` at startup. `BookRepositoryImpl__AotRepository` already contains it. To write this SQL at build time, Spring Data needs the database dialect. Normally Boot asks the database, but there is no database during the build:

```

@Configuration(proxyBeanMethods = false)
class JdbcDialectConfiguration {

    @Bean
    JdbcPostgresDialect jdbcDialect() {
        return JdbcPostgresDialect.INSTANCE;
    }
}

```

Without this bean, `process-aot` fails with `Failed to determine a suitable driver class: it tries to create the DataSource to find the dialect.`

! Important

`process-aot` starts the application context at build time (bean definitions only, without starting the beans). Everything that decides which beans exist, such as profiles and `@ConditionalOnProperty`, is fixed at build time. A profile that you switch on at runtime cannot add beans to an AOT-processed application.

AotProcessingIT runs after package and checks the generated files. To run the jar on the JVM with the generated code, add `-Dspring.aot.enabled=true`.

3.3 Where AOT Is Not Enough: Dynamic Code

Two parts of the application are dynamic on purpose. The price format is configured by class name (`bookstore.price-format.class-name`) and created by reflection:

```
private static PriceFormat instantiate(String className) {
    try {
        Class<?> type = Class.forName(className);           // invisible for
        ↪ static analysis
        return (PriceFormat) type.getDeclaredConstructor().newInstance();
    } catch (ReflectiveOperationException e) {
        throw new IllegalStateException("Cannot create price format " + className, e);
    }
}
```

QuoteOfTheDay reads `quotes/quotes.txt` from the classpath. On the JVM both always work. In a native image, `TurkishLiraFormat` would be removed (no code calls its constructor), and `quotes.txt` would not be in the image.

3.4 RuntimeHints

```
public class BookstoreRuntimeHints implements RuntimeHintsRegistrar {

    @Override
    public void registerHints(RuntimeHints hints, @Nullable ClassLoader classLoader) {
        // PriceFormats creates these by name: keep the classes and allow calling their
        ↪ constructors
        hints.reflection()
            .registerType(TurkishLiraFormat.class,
                ↪ MemberCategory.INVOKE_DECLARED_CONSTRUCTORS)
            .registerType(EuroFormat.class,
                ↪ MemberCategory.INVOKE_DECLARED_CONSTRUCTORS);
        // QuoteOfTheDay reads this file: include it in the image
        hints.resources().registerPattern("quotes/*.txt");
    }

    @Configuration(proxyBeanMethods = false)
    @ImportRuntimeHints(BookstoreRuntimeHints.class)
    static class Registration {
    }
}
```

Hints are plain data, so a unit test can check them without building an image:

```
@Test
void theHintsCoverTheReflectionAndTheResources() {
    RuntimeHints hints = new RuntimeHints();
    new BookstoreRuntimeHints().registerHints(hints, getClass().getClassLoader());

    assert-
        ↪ That(RuntimeHintsPredicates.reflection().onType(TurkishLiraFormat.class)).accepts(hints);
    assert-
        ↪ That(RuntimeHintsPredicates.reflection().onType(EuroFormat.class)).accepts(hints);
    assert-
        ↪ That(RuntimeHintsPredicates.resource().forResource("quotes/quotes.txt")).accepts(hints);
}
```

Other ways to give hints:

Annotation	For
@ImportRuntimeHints(MyHints.class)	registers a RuntimeHintsRegistrar (as above)
@RegisterReflectionForBinding(Dto.class)	types that you (de)serialize yourself, for example with JsonMapper
@Reflective	a method or class that is called by reflection

★ Tip

If a native image fails with `ClassNotFoundException`, `NoSuchMethodException` or a missing resource, you need a hint. The GraalVM tracing agent (`-agentlib:native-image-agent`) can record the metadata while the application runs on the JVM, as a starting point.

3.5 Building the Native Image

Spring Boot's native profile runs `process-aot` and prepares the jar. Paketo buildpacks then run GraalVM (BellSoft Liberica NIK 25) inside Docker:

```
./mvnw -Pnative -pl modules/19-native-performance/lesson spring-boot:build-image
```

The image name and the Java version for the buildpack are set in the `process-aot` snippet above (`BP_JVM_VERSION=25`). The first build downloads the builder and GraalVM and takes several minutes. `native-image` needs about 4 GB of free memory inside Docker.

! Warning

If the build stops with The Native Image build process ran out of memory (exit status 137), other containers use Docker's memory. Stop the services you do not need, for example `docker compose --profile all stop elasticsearch kafka lgtm mongo hazelcast redis`, or give Docker more memory.

Start the native image against the compose PostgreSQL:

```
docker run --rm -p 8080:8080 springbootedu/native-performance:1.0.0-SNAPSHOT \
  --spring.datasource.url=jdbc:postgresql://host.docker.internal:5432/bookstore \
  --spring.datasource.username=bookstore --spring.datasource.password=bookstore
```

```
=== mode: native image, ready in 173 ms ===
Designing Data-Intensive Applications - ₺110,00
```

The first native build of this lesson printed ₺110.00 instead of ₺110,00: the reflection hint worked, but a native image contains only the default locale (en). The Turkish and German number formats need the locales at build time. Build options that belong to the application go into a `native-image.properties` file in `META-INF/native-image/`, so they work for buildpacks, `native:compile` and CI alike:

```
Args = -H:IncludeLocales=tr-TR,de-DE
```

With a local GraalVM 25 (GRAALVM_HOME) and Maven on JDK 27, `./mvnw -Pnative -pl modules/19-native-performance/lesson native:compile` builds an executable for your own operating system. The CI workflow `native.yml` does exactly this on Linux and smoke-tests the executable. It is started by hand, because it takes several minutes.

3.6 The JVM AOT Cache and the Comparison

Since JDK 24 (JEP 483), the JVM can store loaded and linked classes in an **AOT cache**. JDK 25 made it a one-step command (JEP 514) and added method profiles (JEP 515). A **training run** starts the application and writes the cache. Spring Boot helps with `-Dspring.context.exit=onRefresh`: the application stops right after the context is ready.

```
java -XX:AOTCacheOutput=app.aot -Dspring.aot.enabled=true
↪ -Dspring.context.exit=onRefresh -jar app.jar
java -XX:AOTCache=app.aot -Dspring.aot.enabled=true -jar app.jar
```

The cache works only with the same JDK and the same classpath. Boot therefore recommends an extracted jar: `java -Djarmode=tools -jar app.jar extract --destination application`.

`compare-startup.sh` does all of this and measures every variant (median of several runs):

```
modules/19-native-performance/compare-startup.sh 5
```

Variant (median of 5 runs)	Start ms	RSS MB
JVM	1716	296
JVM + Spring AOT	1623	267
JVM + Spring AOT + AOT cache	736	208
Native image (GraalVM 25)	226	71

- **Spring AOT alone** saves little here: the context of a small application is quick to build anyway. It matters more with many beans, and it is the precondition for a native image.
- **The AOT cache** more than halves the start: class loading and linking, the largest part of a JVM start, happen in the training run.
- **The native image** starts about 7× faster than the plain JVM and needs about a quarter of the memory.

“Start” is Boot’s “process running for” value, so it includes the JVM’s own start. The native memory is measured by `docker stats` and the JVM memory is the process RSS. The numbers are from a MacBook (Apple silicon) and are different on your machine: compare the proportions, not the milliseconds.

4. Common Mistakes and Best Practices

!! Caution

A native image that builds is not a native image that works. Dynamic code fails only when it runs. Test the native executable (the CI smoke test calls the reflective and resource endpoints), or run the tests natively with `-PnativeTest`.

- **Do:** test your `RuntimeHints` with `RuntimeHintsPredicates`, like any other code.
- **Don’t:** expect runtime profiles or `@ConditionalOnProperty` to change the beans of an AOT-processed application. Decide them at build time.
- **Do:** measure before and after. The AOT cache is almost free (a training run and one JVM flag). A native image changes the build, the debugging and the libraries you can use.
- **Don’t:** compare a native image with a JVM on start time only. After warm-up, the JIT compiler of the JVM often reaches a higher throughput than a native image.
- **Do:** keep the training run close to real use, so that the cache contains the classes that production needs.

5. Summary

- Spring AOT moves work from startup to build time: bean definitions and repository implementations become generated code, and the hints become GraalVM metadata.
- AOT processing runs without a database, so settings like the JDBC dialect must be explicit.
- Code that is reached dynamically needs RuntimeHints (reflection, resources, binding), and these hints can be unit tested.
- Paketo buildpacks build a GraalVM 25 native image inside Docker. A native image starts in milliseconds and uses little memory, but gives up build speed and some dynamism.
- The JVM AOT cache (Project Leyden) makes a normal JVM start much faster with one training run.

Next step: [Exercises](#)

6. Further Reading

- [Spring Boot — GraalVM Native Image Support](#)
- [Spring Boot — AOT Cache](#)
- [Spring Framework — Ahead of Time Optimizations](#)
- [Spring Data JDBC — AOT Repositories](#)
- [GraalVM — Reachability Metadata](#)
- [JEP 483: Ahead-of-Time Class Loading & Linking](#) · [JEP 514](#) · [JEP 515](#)